

Conteúdo de Revisão — Aula 03: Ponteiros, Referências e Estruturas Dinâmicas em Python

Estrutura de Dados

Duração: 1h30

Linguagem: Python

Tema: Ponteiros de memória, referências e estruturas dinâmicas.

```
# =====
# PARTE 1: A Ilusão da Memória, Referências e Mutabilidade em Python
# =====

def demonstrar_referencias():
    # Imprime um cabeçalho para a seção de Identidade de Objetos.
    print("--- 1. Identidade de Objetos (id) ---")

    # Cria uma variável 'a' e atribui o valor inteiro 5.
    a = 5
    # Cria uma variável 'b' e atribui o valor de 'a'.
    # Neste ponto, 'b' refere-se ao *mesmo* objeto que 'a' na memória, pois inteiros são imutáveis e pequenos valores são 1
    b = a

    # A função id() retorna o identificador único de um objeto, que geralmente corresponde ao seu endereço de memória.
    # Mostra o valor de 'a' e seu ID.
    print(f"Valor de a: {a} | id(a): {id(a)}")
    # Mostra o valor de 'b' e seu ID. Observará que o ID de 'a' e 'b' são os mesmos.
    print(f"Valor de b: {b} | id(b): {id(b)}")
    # Confirma se ambos os IDs são idênticos, provando que 'a' e 'b' referem-se ao mesmo objeto.
    print(f"a e b apontam para o mesmo objeto? {id(a) == id(b)}\n")

    # Imprime um cabeçalho para a seção de Mutabilidade vs Imutabilidade.
    print("--- 2. Mutabilidade vs Imutabilidade ---")

    # --- Demonstração com Tipo Imutável (Inteiro) ---
    # 'x' recebe o valor 10. Inteiros são tipos imutáveis.
    x = 10
    # Armazena o ID original de 'x'.
    id_x_antes = id(x)
    # Altera o valor de 'x' somando 1. Como inteiros são imutáveis, esta operação
    # não modifica o objeto original '10'. Em vez disso, um *novo* objeto '11' é criado
    # na memória, e 'x' passa a referenciar este novo objeto.
    x = x + 1
    # Compara o ID de 'x' antes e depois da modificação. Eles serão diferentes.
    print(f"x antigo id: {id_x_antes} -> x novo id: {id(x)} (Objeto foi recriado)")

    # --- Demonstração com Tipo Mutável (Lista) ---
    # 'lista1' recebe uma lista. Listas são tipos mutáveis.
    lista1 = [1, 2, 3]
    # Armazena o ID original de 'lista1'.
    id_lista1_antes = id(lista1)
    # Adiciona um elemento à lista usando o método .append().
    # Como listas são mutáveis, esta operação modifica o *conteúdo* do objeto lista
    # diretamente na memória. Nenhum novo objeto lista é criado.
    lista1.append(4)
    # Compara o ID de 'lista1' antes e depois da modificação. Eles permanecerão os mesmos.
    print(f"Lista id antes: {id_lista1_antes} -> Lista id depois: {id(lista1)} (Objeto alterado no mesmo local)\n")

    # Imprime um cabeçalho para a seção de Referência vs Cópia de Listas.
    print("--- 3. Referência vs Cópia de Listas ---")

    # --- Atribuição por Referência (Alias) ---
    # 'lista_original' recebe uma nova lista.
    lista_original = [10, 20, 30]
    # 'lista_referencia' recebe 'lista_original'. Isso não cria uma nova lista.
    # Ambas as variáveis agora apontam para o *mesmo* objeto lista na memória.
    lista_referencia = lista_original

    # --- Criação de uma Cópia Independente (Slice) ---
    # 'lista_copia' é criada usando um 'slice' completo ([:]).
    # Este método cria uma *nova* lista na memória que contém os mesmos elementos da 'lista_original'.
    # 'lista_copia' é um objeto independente.
    lista_copia = lista_original[:]

    # --- Alterando a Lista Original ---
```

```
# Adiciona um elemento à 'lista_original'.
# Como 'lista_referencia' aponta para o mesmo objeto, ela também refletirá essa alteração.
# 'lista_copia', sendo independente, não será afetada.
lista_original.append(99)

# Imprime os resultados para cada lista, incluindo seus IDs.
# 'lista_original' mostrará o novo elemento e terá um ID.
print(f"Original:  {lista_original} | id: {id(lista_original)}")
# 'lista_referencia' mostrará o novo elemento e terá o MESMO ID que 'lista_original'.
print(f"Referência: {lista_referencia} | id: {id(lista_referencia)} (Sofreu alteração!)")
# 'lista_copia' NÃO mostrará o novo elemento e terá um ID DIFERENTE dos outros.
print(f"Cópia:      {lista_copia} | id: {id(lista_copia)} (Permaneceu isolada)")

# Chama a função para executar todas as demonstrações quando o script é rodado.
demonstrar_referencias()
```

```
--- 1. Identidade de Objetos (id) ---
Valor de a: 5 | id(a): 11278056
Valor de b: 5 | id(b): 11278056
a e b apontam para o mesmo objeto? True

--- 2. Mutabilidade vs Imutabilidade ---
x antigo id: 11278216 -> x novo id: 11278248 (Objeto foi recriado)
Lista id antes: 137246164338624 -> Lista id depois: 137246164338624 (Objeto alterado no mesmo local)

--- 3. Referência vs Cópia de Listas ---
Original:  [10, 20, 30, 99] | id: 137246164078720
Referência: [10, 20, 30, 99] | id: 137246164078720 (Sofreu alteração!)
Cópia:     [10, 20, 30] | id: 137246162729792 (Permaneceu isolada)
```

Exercício Prático: Sistema de Gerenciamento de Clínica

✓ 18. Problema real — Sistema de atendimento de uma clínica

Uma clínica recebe pacientes durante o dia. Cada paciente possui:

- nome;
- idade;
- prioridade.

Agora iremos aplicar os conceitos de referências criando uma Estrutura Dinâmica (Lista Encadeada / Fila).

Classe Paciente: Representa os dados guardados em cada Nó.

Classe Node: Contém o dado e a referência (proximo) para o próximo nó.

Classe FilaClinica: Gerencia os apontadores inicio e fim da fila.

Desafio: Implementação da inserção com Prioridade (Idade $\geq$ 60 anos).

```
# =====
# PARTE 2: Construção Dinâmica - Nó e Fila de Atendimento da Clínica
# =====

class Paciente:
    """Classe responsável por armazenar as informações de cada paciente."""
    def __init__(self, nome: str, idade: int):
        self.nome = nome
        self.idade = idade
        # Define se é prioritário com base na idade
        self.eh_prioritario = idade >= 60

    def __repr__(self):
        status = "PRIORITÁRIO" if self.eh_prioritario else "NORMAL"
        return f"[{self.nome}, {self.idade} anos - {status}]"

class Node:
    """A estrutura básica de um Nó dinâmico contendo o dado e a referência do próximo."""
    def __init__(self, paciente: Paciente):
        self.dado = paciente # Armazena a instância do paciente
        self.proximo = None # Âncora/Referência para o próximo nó na corrente

class FilaClinica:
    """
    Gerenciador da Fila de Pacientes.
    Manipula as referências 'inicio' e 'fim' para adicionar e remover de forma eficiente.
    """
    def __init__(self):
        self.inicio = None
        self.fim = None
```

```

self._tamanho = 0

def esta_vazia(self) -> bool:
    """Verifica se a fila possui pacientes."""
    return self.inicio is None

def adicionar(self, nome: str, idade: int):
    """
    Adiciona um paciente respeitando as regras de prioridade:
    - Prioritários (idade >= 60) entram antes dos normais, respeitando a ordem de chegada.
    - Não-prioritários vão sempre para o final da fila.
    """
    novo_paciente = Paciente(nome, idade)
    novo_no = Node(novo_paciente)

    # CASO 1: Fila está totalmente vazia
    if self.esta_vazia():
        self.inicio = novo_no
        self.fim = novo_no
        self._tamanho += 1
        print(f"-> Paciente {novo_paciente.nome} adicionado como o primeiro da fila.")
        return

    # CASO 2: Paciente é PRIORITÁRIO
    if novo_paciente.eh_prioritario:
        # Subcaso 2.1: O primeiro paciente da fila é Normal -> Novo nó entra no início
        if not self.inicio.dado.eh_prioritario:
            novo_no.proximo = self.inicio
            self.inicio = novo_no
        else:
            # Subcaso 2.2: Percorre a fila para encontrar o último prioritário
            atual = self.inicio
            while atual.proximo is not None and atual.proximo.dado.eh_prioritario:
                atual = atual.proximo

            # Reorganiza os ponteiros/referências para encaixar o nó prioritário
            novo_no.proximo = atual.proximo
            atual.proximo = novo_no

            # Se foi inserido no final da fila, atualiza a referência do fim
            if novo_no.proximo is None:
                self.fim = novo_no

    # CASO 3: Paciente NÃO PRIORITÁRIO (Insere sempre no final)
    else:
        self.fim.proximo = novo_no
        self.fim = novo_no

    self._tamanho += 1
    print(f"-> Paciente {novo_paciente.nome} ({novo_paciente.idade} anos) adicionado à fila.")

def atender(self):
    """Remove e retorna o primeiro paciente da fila (atendimento)."""
    if self.esta_vazia():
        print("\n⚠ A fila está vazia! Nenhum paciente para atender.")
        return None

    # O primeiro da fila é selecionado
    paciente_atendido = self.inicio.dado
    # Move o ponteiro 'inicio' para o próximo da fila
    self.inicio = self.inicio.proximo
    self._tamanho -= 1

    # Se a fila ficou vazia após o atendimento, o fim também vira None
    if self.inicio is None:
        self.fim = None

    print(f"\n✅ Atendendo: {paciente_atendido.nome}")
    return paciente_atendido

def listar_espera(self):
    """Percorre a fila do início ao fim exibindo todos os pacientes."""
    print("\n--- 📁 FILA DE ESPERA ATUAL ---")
    if self.esta_vazia():
        print("Nenhum paciente aguardando.")
        print("-----\n")
        return

    atual = self.inicio
    posicao = 1

    # IMPORTANTE: A variável 'atual' caminha para frente a cada iteração (atual = atual.proximo)
    # para evitar o ERRO CRÍTICO de Loop Infinito

```

```

    # para evitar o erro de acesso de dados em branco:
    while atual is not None:
        print(f"{posicao}º -> {atual.dado}")
        atual = atual.proximo # Avança para o próximo nó
        posicao += 1
    print(f"Total na fila: {self._tamanho}")
    print("-----\n")

```

```

# =====
# PARTE 3: Execução e Testes do Sistema de Atendimento
# =====

# Instanciando a fila da clínica
clinica = FilaClinica()

print("=== 1. CHEGADA DE PACIENTES NORMIAIS E PRIORITÁRIOS ===")
clinica.adicionar("João", 30) # Normal
clinica.adicionar("Maria", 25) # Normal
clinica.adicionar("Vovó Ana", 72) # Prioritário (Deve ir para a frente da fila)
clinica.adicionar("Pedro", 40) # Normal
clinica.adicionar("Vovô Bento", 80) # Prioritário (Deve ir atrás da Vovó Ana, mas antes dos normais)

# Exibe o estado da fila
clinica.listar_espera()

print("=== 2. ATENDIMENTO DOS PACIENTES ===")
# Atendendo o primeiro prioritário (Vovó Ana)
clinica.atender()
clinica.listar_espera()

# Atendendo o segundo prioritário (Vovô Bento)
clinica.atender()
clinica.listar_espera()

print("=== 3. ATENDENDO O RESTANTE DOS PACIENTES ===")
clinica.atender() # João
clinica.atender() # Maria
clinica.atender() # Pedro

# Tentando atender com fila vazia
clinica.atender()
clinica.listar_espera()

```

```

=== 1. CHEGADA DE PACIENTES NORMIAIS E PRIORITÁRIOS ===
-> Paciente João adicionado como o primeiro da fila.
-> Paciente Maria (25 anos) adicionado à fila.
-> Paciente Vovó Ana (72 anos) adicionado à fila.
-> Paciente Pedro (40 anos) adicionado à fila.
-> Paciente Vovô Bento (80 anos) adicionado à fila.

```

```

--- 📄 FILA DE ESPERA ATUAL ---
1º -> [Vovó Ana, 72 anos - PRIORITÁRIO]
2º -> [Vovô Bento, 80 anos - PRIORITÁRIO]
3º -> [João, 30 anos - NORMAL]
4º -> [Maria, 25 anos - NORMAL]
5º -> [Pedro, 40 anos - NORMAL]
Total na fila: 5
-----

```

```

=== 2. ATENDIMENTO DOS PACIENTES ===

```

```

✅ Atendendo: Vovó Ana

```

```

--- 📄 FILA DE ESPERA ATUAL ---
1º -> [Vovô Bento, 80 anos - PRIORITÁRIO]
2º -> [João, 30 anos - NORMAL]
3º -> [Maria, 25 anos - NORMAL]
4º -> [Pedro, 40 anos - NORMAL]
Total na fila: 4
-----

```

```

✅ Atendendo: Vovô Bento

```

```

--- 📄 FILA DE ESPERA ATUAL ---
1º -> [João, 30 anos - NORMAL]
2º -> [Maria, 25 anos - NORMAL]
3º -> [Pedro, 40 anos - NORMAL]
Total na fila: 3
-----

```

```

=== 3. ATENDENDO O RESTANTE DOS PACIENTES ===

```

```

✅ Atendendo: João

```

```

✅ Atendendo: Maria

```

✅ Atendendo: Pedro

⚠️ A fila está vazia! Nenhum paciente para atender.

--- 📄 FILA DE ESPERA ATUAL ---
Nenhum paciente aguardando.
